What is....

Pipeline RAG pour les actualités en temps réel

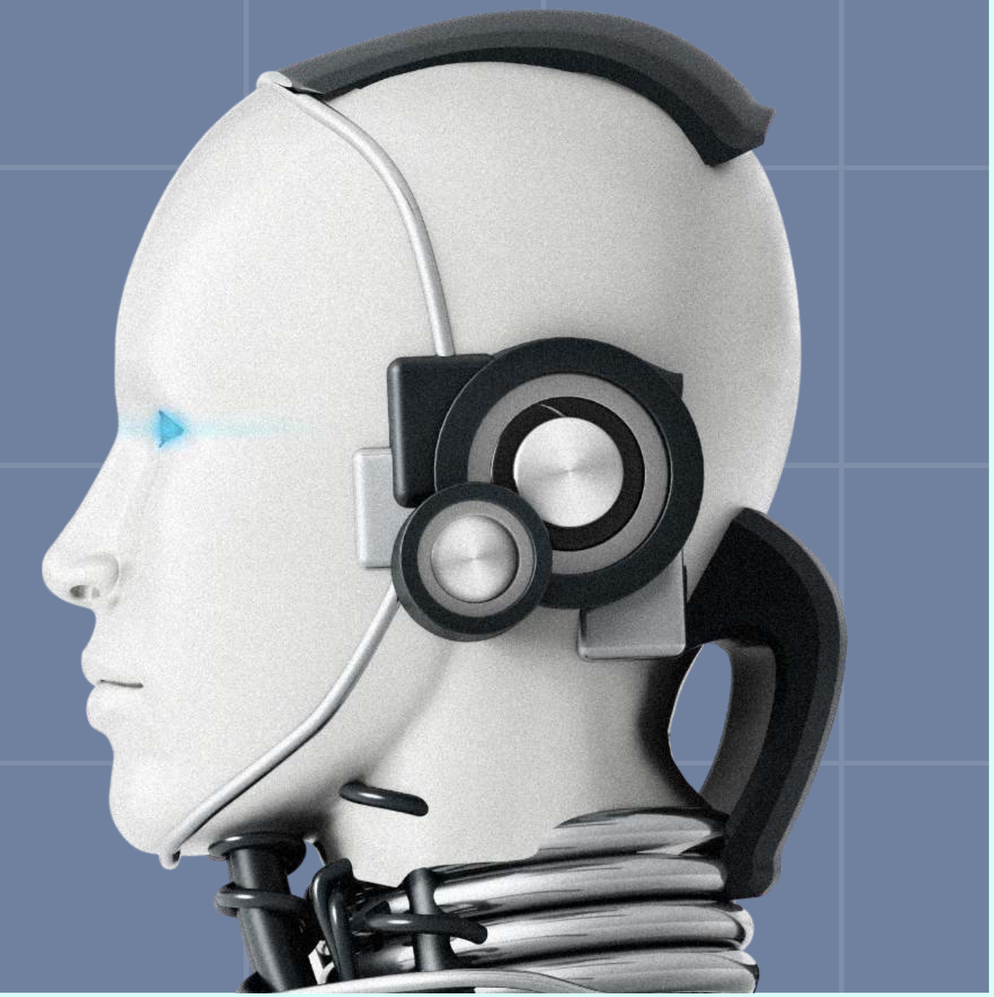
Présenté par :

Aya Taki

Hajar Braidi



Plan



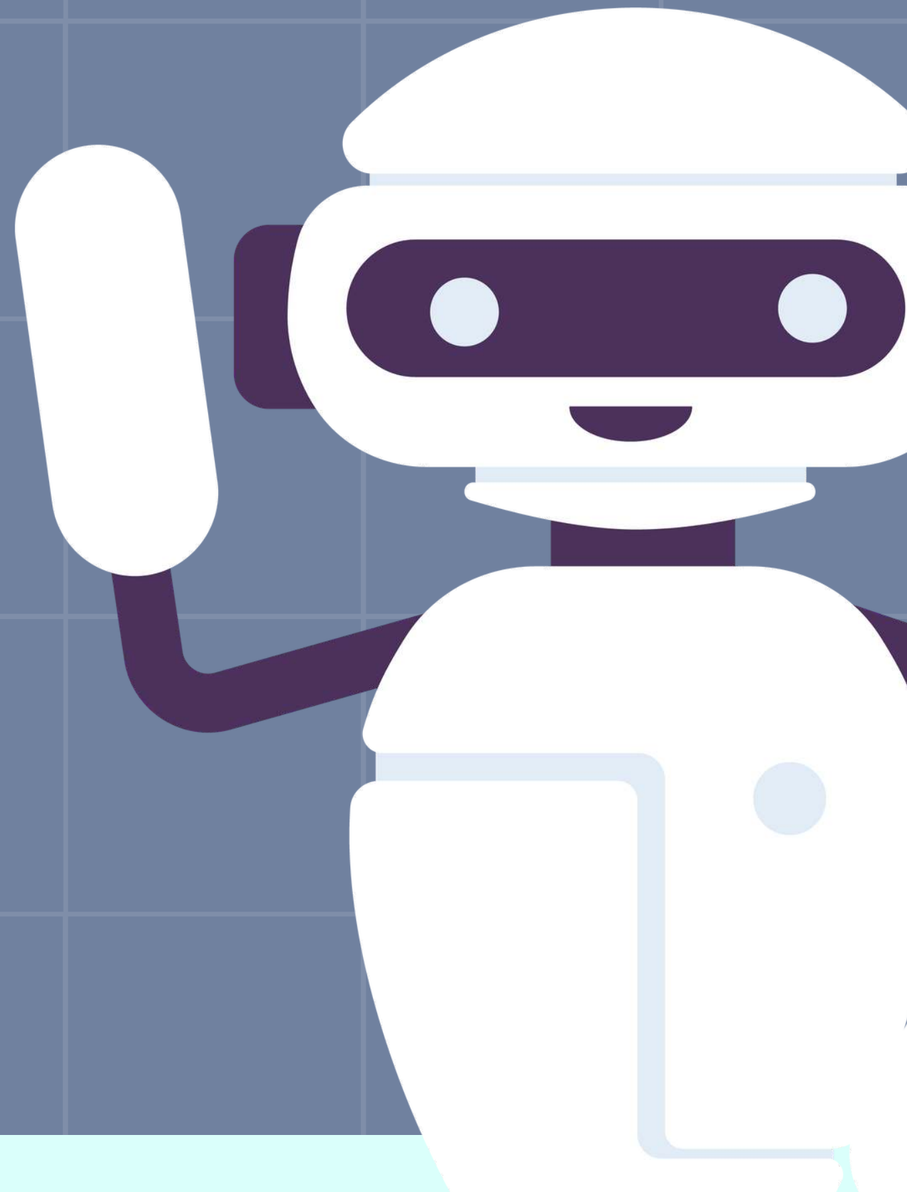
Introduction

1. Problématique & objectifs
2. Architecture du pipeline
3. Couche d’ingestion
4. Couche de stockage
5. Service d’embedding
6. Service RAG
7. Infrastructure Docker & GitHub
8. Démonstration & Métriques

Conclusion

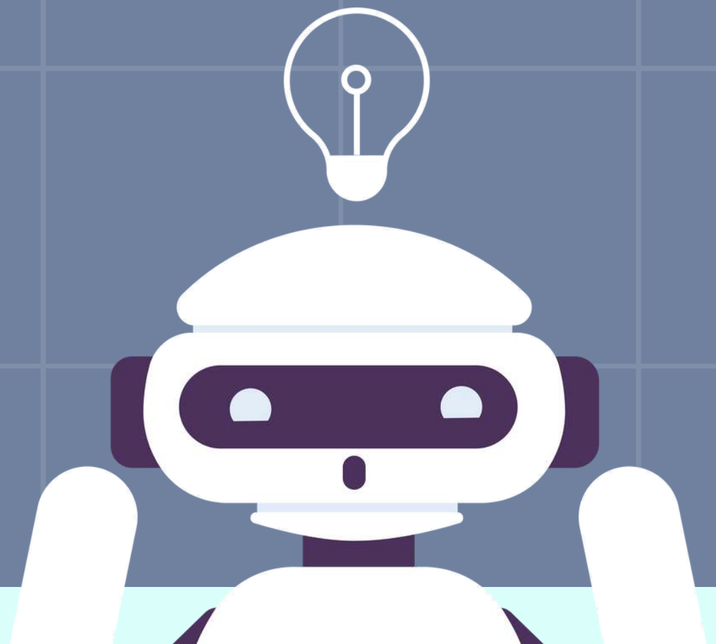
Introduction

La RAG (Génération à enrichissement contextuel) est une technique économique qui améliore les réponses d'un grand modèle de langage (LLM) en le connectant à une base de connaissances externe et fiable (données spécifiques ou internes à une entreprise).



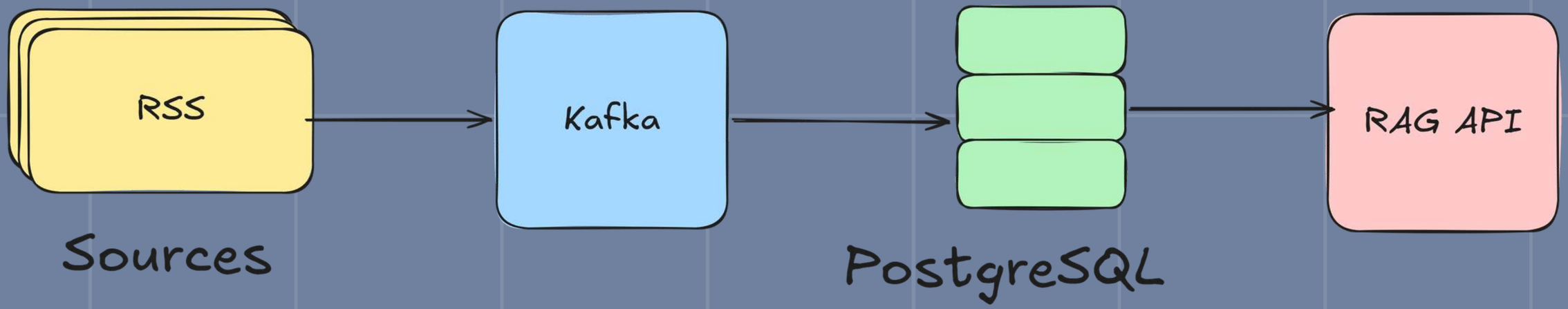
Problématique & objectifs

- L’actualité marocaine est fragmentée sur plusieurs sources.
- Les LLMs n’ont pas accès aux données récentes, ce qui induit des halucinations.
- Impossible d’intéroger +1000 articles manuellement.
- Les réponses génériques ne citent pas de sources vérifiables.



- Ingestion automatique.
- RAG en temps réel.
- Recherche sémantique.
- Réponses sourcées.

Architecture du pipeline



Le Matin
Aujourd'hui
La nouvelle tribune

3 partitions
Rétention de 24h

articles
article_embeddings

FastAPI
/ask + /search

Couche d’ingestion



3 sources toutes les 5 minutes

feedparser BeautifulSoup

news.raw 3 partitions

Couche de stockage

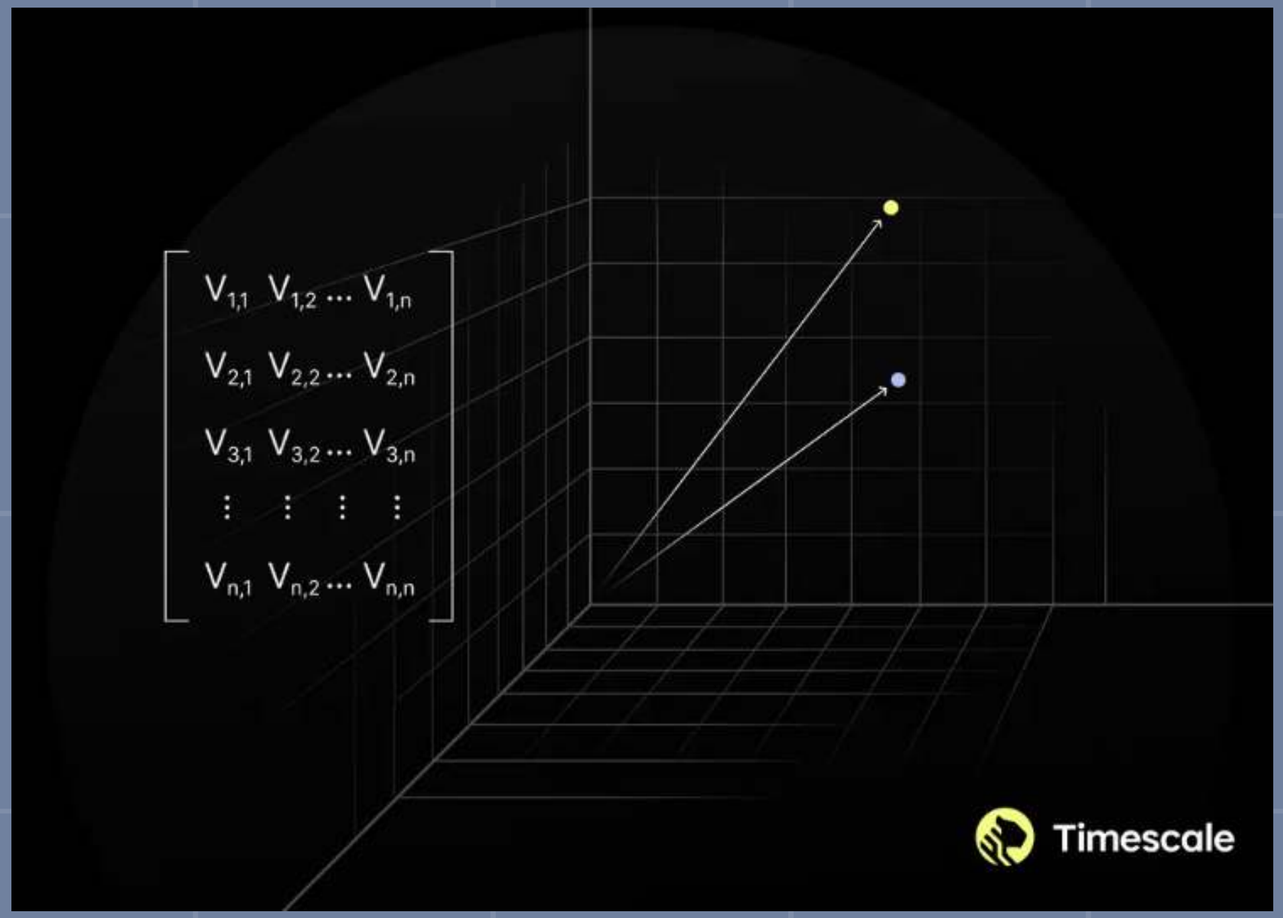
PostgreSQL + pgvector

Table articles	
article_id	Varchar(64) PK
source_name	Varchar(128) - journal src
title	TEXT - cleaned title
content	TEXT - whole content
url	TEXT UNIQUE
published_at	TIMESTAMP

Table article_embeddings	
article_id	FK - articles
embedding	vector(384)
model_name	Varchar - used model
embedded_at	TIMESTAMP

Service d’embedding

Vectorisation des articles



https://www.tigerdata.com/blog/a-beginners-guide-to-vector-embeddings

Service d’embedding

Intfloat/multilingual-e5-small

384 dimensions - compacts, performant

120 Mo - tourne sur CPU

50 langues - FR + AR + EN

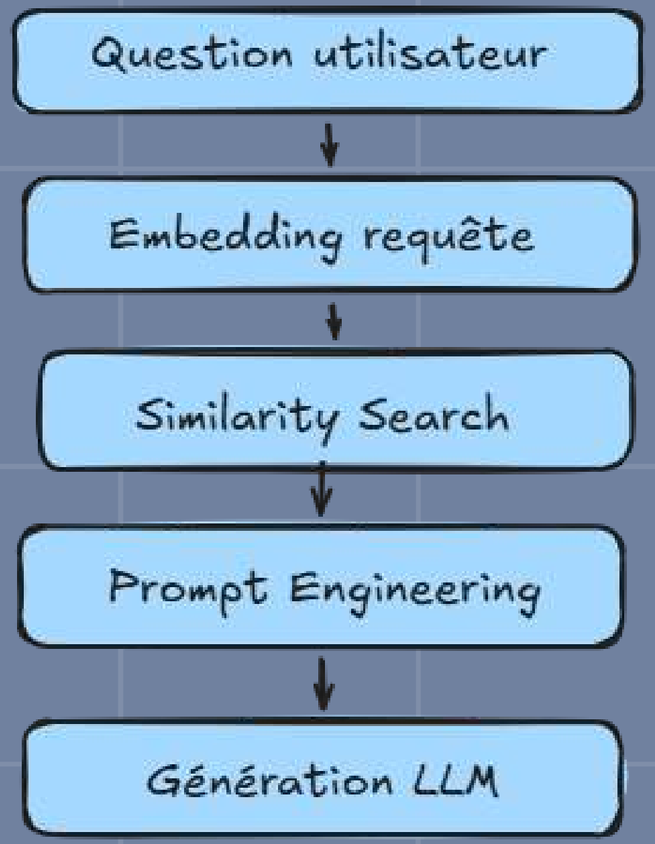
Asymétrique - query / passage

Boucle de vectorisation

1. Fetch des articles non vectorisés
2. Build text: title + desc + content[:2000]
3. Préfixe “passage :” + encode ()
4. Sleep 60s si aucun article en attente

Service RAG

Pipeline de génération



POST /ask — min 5 chars, max 500

On préfixe la question avec query => précision de 15 à 20%

HNSW _ pgvector <=> cosine _ threshold 0.30

5 articles en contexte · template structuré · citation sources

Gemma 3 via Ollama · temperature=0.3 · réponses factuelles.

Infrastructure

Docker & Github Codespace

articles-db:
pgvector:pg15
(Port 5432).

rss-producer:
Polling continu.

kafka & zookeeper:
Broker 3 partitions & Coordination.

consumer:
Pont Kafka vers PostgreSQL.

embedding-service:
CPU-only PyTorch.

rag-service:
FastAPI & Streaming SSE
(Port 8000).

Déployement cloud entièrement conteneurisé (4 cœurs · 8 GB RAM · 32 GB disque)

Démonstration

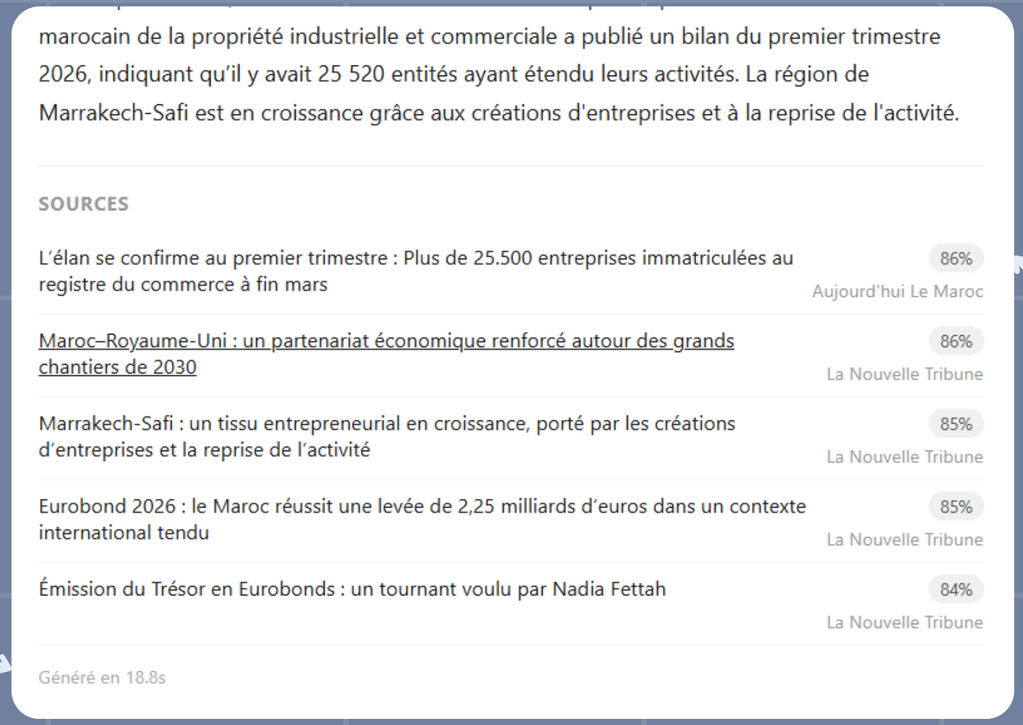
Interface minimaliste interrogeant l'API via /ask/stream.

Réponse fluide en Server-Sent Events (SSE) intégrant des données factuelles (ex: 9.133 entreprises, 122 milliards de dirhams).

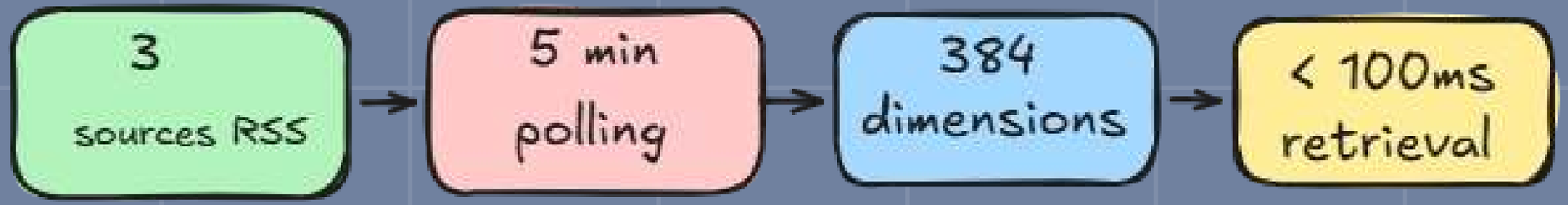


Résultats & Métriques

Temps d'execution total affiche en secondes pour évaluer la latence globale.

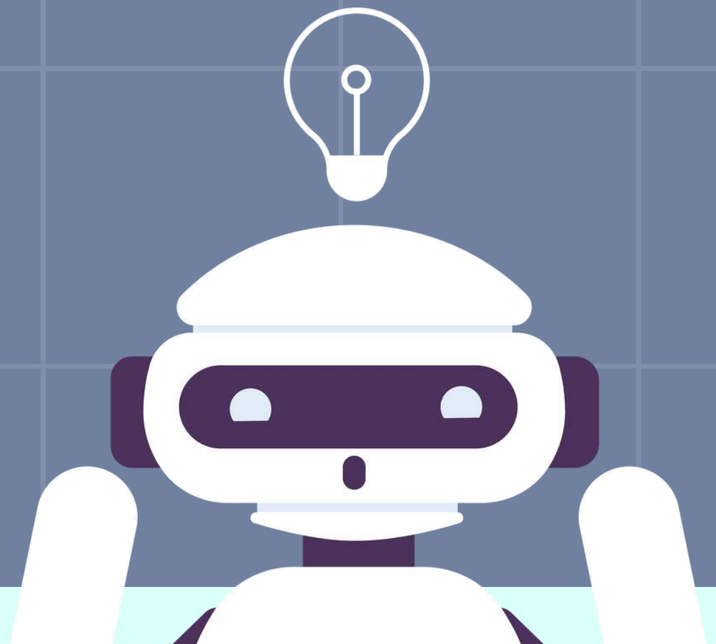


Affichage transparent des scores de similarite vectorielle (ex: 85%, 86%).



Conclusion & Perspectives

- Construction d’un pipeline complet et fonctionnel, de l'ingestion RSS jusqu'à la réponse générée, entièrement conteneurisé.



- Passer à un GPU pour réduire le temps de génération.
- Ajouter d'autres sources RSS marocaines.
- Tester un modèle plus grand comme Gemma 3 4B pour des réponses plus riches.

**

Merci

